

EECE 571Q

## **Lecture 04: Just stay still!**

---

Thursday 21 May 2026

# Announcements

- Classes on Zoom next week - link in Piazza
- Quiz 2 on Tuesday

# Last time

We defined conditions for quantum error correction

“( $Q, \mathcal{E}$ ) is a QECC iff  $\forall |\psi\rangle, |\phi\rangle \in Q, \forall E_a, E_b \in \mathcal{E}$ ,

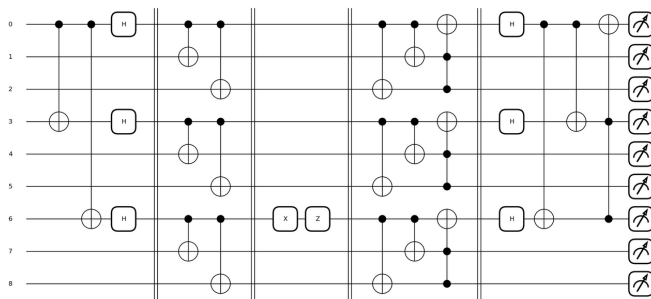
$$\langle \psi | E_a^\dagger E_b | \phi \rangle = C_{ab} \langle \psi | \phi \rangle,$$

Note that  $C_{ab}$  does not depend on  $|\psi\rangle$  or  $|\phi\rangle$ ”

Source: Gottesman, Thm. 2.7

## Last time

We combined the bit and phase flip codes into a 9-qubit code that can correct one arbitrary single-qubit error.



# Learning outcomes

- 1 Express quantum error correcting codes in the stabilizer formalism
- 2 Characterize whether Pauli errors are correctable by a code using its stabilizer generators
- 3 Define classical linear codes, and formulate them in terms of generator and parity check matrices
- 4 Use classical linear codes to construct CSS codes for quantum error correction

# Stabilizers

Let  $T$  be a QECC. The **stabilizer** of the code,  $S(T)$ , is

In words: a (sub)group of Paulis for which all codewords are  $+1$  eigenstates.

Facts about  $S(T)$ :

- $-I$  is never in  $S(T)$
- all elements of  $S(T)$  commute (Abelian subgroup of  $\mathcal{P}_n$ )
- choosing a subgroup  $S$  defines the stabilized codewords of  $T$

# Stabilizer codes

More convenient to work “backwards”: given  $S$ , define codespace

If  $T = \mathcal{T}(S(T))$ ,  $T$  is a **stabilizer code**.

# Bit flip code revisited

An alternative way of thinking about the parity computation is in terms of eigenvalues w.r.t. the code's stabilizer group.

Consider our logical states:

Which three-qubit Paulis are these states  $+1$  eigenstates of?



# Bit flip code revisited

Let's determine a *minimal* representation of the group in terms of its **stabilizer generators**.

**Exercise:** if there are  $r$  stabilizer generators, how large is the stabilizer group  $S$ ?

# Stabilizers generators

We can also work backwards to obtain codewords from the stabilizer generators by defining a projector onto the codespace,

$\Pi_S$  provides insight into size of the stabilized space: each generator cuts space in half, so  $r$  generators gives space with dimension  $2^{n-r}$ .

Special case: if  $r = n$ , have a single state called a *stabilizer state*.

# Bit flip code: stabilizers and logical operations

Logical  $Z$ : need a Pauli that

- stabilizes the logical state but is *not* in  $S$
- *commutes* with everything in  $S$
- has the right action on the subspace

Logical  $X$ : need a Pauli that

- stabilizes the logical state but is *not* in  $S$
- *commutes* with everything in  $S$
- has the right action on the subspace
- anticommutes with logical  $Z$

# Phase flip code: stabilizer formalism

# Stabilizer measurement and error detection

We can use this formalism to construct circuits for error detection and recovery: simply *measure the stabilizer generators*.

Correctable errors anticommute with at least one generator. We can detect their presence in the syndrome measurement.

# Stabilizers and logical errors

Danger: error that *commutes* with all elements of  $S$ , but isn't in  $S$ .

More precisely, define the *normalizer*  $N(S)$  as

For an arbitrary error  $E$ ...

- if  $E \notin N(S)$ ,  $E$  is detectable
- if  $E \in N(S)$  and  $E \in S$ , the code space is unchanged
- if  $E \in N(S)$  but  $E \notin S$ ,  $E$  is **undetectable**

# Stabilizers and logical errors

Gottesman Thm 3.4

“The set of undetectable errors for a stabilizer code  $S$  is  $\hat{N}(S) \setminus \hat{S}$ . The **distance** of  $S$  is  $\min\{\text{wt}(E) \mid \hat{E} \in \hat{N}(S) \setminus \hat{S}\}$ ”.

Gottesman Thm 3.5

“The stabilizer code  $S$  corrects a set of errors  $\mathcal{E} \subseteq \mathcal{P}_n$  iff  $\hat{E}^\dagger \hat{F} \notin \hat{N}(S) \setminus \hat{S}$  for all  $E, F \in \mathcal{E}$ ”.

# Shor code: stabilizer formalism

Stabilizer code usually described by notation  $[[n, k, d]]$ . The 9-qubit Shor code is a  $[[9, 1, 3]]$  code.

| Name      | Operator      |
|-----------|---------------|
| $g_1$     | $ZZIIIIII$    |
| $g_2$     | $IZZIIIIII$   |
| $g_3$     | $III ZZ III$  |
| $g_4$     | $IIII ZZ III$ |
| $g_5$     | $IIIIII ZZ I$ |
| $g_6$     | $IIIIII ZZ$   |
| $g_7$     | $XXXXXX III$  |
| $g_8$     | $III XXXXXX$  |
| $\bar{Z}$ | $XXXXXXXXXX$  |
| $\bar{X}$ | $ZZZZZZZZZZ$  |

What is an example of a weight-3 undetectable error?

Screenshot: Nielsen & Chuang, chapter 10.5.6.



# Syndromes and decoding

**Exercise:** What are the error syndromes for  $Y_1$  and  $X_0Z_1X_2$ ?

| Name      | Operator      |
|-----------|---------------|
| $g_1$     | $ZZIIIIIIII$  |
| $g_2$     | $IZZIIIIIIII$ |
| $g_3$     | $IIIZZIIIIII$ |
| $g_4$     | $IIIIIZZIIII$ |
| $g_5$     | $IIIIIIIZZII$ |
| $g_6$     | $IIIIIIIIIZZ$ |
| $g_7$     | $XXXXXXIIII$  |
| $g_8$     | $IIIXXXXXXX$  |
| $\bar{Z}$ | $XXXXXXXXXX$  |
| $\bar{X}$ | $ZZZZZZZZZZ$  |

Screenshot: Nielsen & Chuang, chapter 10.5.6.

# Syndromes and decoding

Two errors,  $E$  and  $F$  are in the same **coset** of  $N(S)$  if  $F = EN$  for some  $N \in N(S)$ . Errors in the same coset of  $N(S)$  have the same error syndrome.

We will explore this more when we discuss decoding.

# Where do codes come from?

We have seen the bit flip, phase flip codes, and 9-qubit Shor code. They are all related.

- Are there better codes?
- Are there smaller codes?
- How do we find new ones?

# Classical codes

Many QECCs are constructed from *classical* error correcting codes.

Source: Gottesman, Def. 4.1

"A classical error-correcting code  $(e, \mathcal{E})$  is a **map**  $e : [1 \dots K] \rightarrow [1 \dots M]$  with the set of **correctable errors**  $\mathcal{E}$  (consisting of functions  $E : [1 \dots M] \rightarrow [1 \dots M]$ ) with the following property:  $\exists$  function  $d : [1 \dots M] \rightarrow [1 \dots K]$  such that  $\forall E \in \mathcal{E}, \forall x \in [1 \dots K],$ "

$$d(E(e(x))) = x$$

# Classical codes

The *distance* of a code  $C \subseteq \mathbb{Z}_N$  is

Consequence: a distance  $d$  code

- corrects general errors on up to  $\lfloor (d-1)/2 \rfloor$  coordinates
- correct erasure errors on up to  $d-1$  coordinates
- detect general errors on up to  $d-1$  coordinates

Notation:  $(n, K, d)$  code

# Classical linear codes

Source: Gottesman, Def. 4.3, section 4.2

A code  $C \subseteq \mathbb{Z}_2^n$  is a *linear code* if

Codewords are linear combinations of basis states for (sub)space  $C$ .

Example: repetition code.  $k = 1$  basis state,  $2^k = 2$  codewords.

# Classical linear codes

Given  $k$  basis vectors  $x_1, \dots, x_k$ , define a  $k \times n$  **generator matrix**

The linear map  $v \in \mathbb{Z}_2^k \rightarrow G_C^T v \in \mathbb{Z}_2^n$  is the *encoder* for  $C$ .

Example: repetition code.

# Classical linear codes

Given  $G_C$ , a  $(n - k) \times n$  **parity check matrix**  $H_C$  contains rows  $y_i$  such that  $Gy_i = 0$ , with  $\{y_i\}$  a maximal linearly independent set.

Relationship between  $G_C$  and  $H_C$ : .

Example: repetition code.



# Classical linear codes

How to model errors?

Apply the parity check matrix...

Example: repetition code.

# Classical codes

(Source: Gottesman Ch. 4.2)

The *distance* of a code  $C \subseteq \mathbb{Z}_N$  is

$$\min\{\text{wt}(E) \mid \exists x \neq y \in C, Ex = y\}$$

The *distance* of a linear code  $C$  is

A linear code  $C$  corrects error set  $\mathcal{E}$

■ iff

■

# The Hamming code

Suppose we wish to correct a 1-bit error on  $r$ -bit code words. Need all error syndromes to be unique.

Code parameters:

General parameters:

# Dual codes

Given code  $C$ , its dual code  $C^\perp$  is

The dual code exchanges roles of  $G$  and  $H$ .

If  $C$  has code parameters  $[n, k, d]$ ,  $C^\perp$  has parameters  $[n, n - k, d]$ .

# Relationship with stabilizer codes

Binary symplectic representation: isomorphism between  $\hat{\mathcal{P}}_n$  and vectors in  $\mathbb{Z}_2^n \times \mathbb{Z}_2^n$ .

Commutator computed from symplectic inner product:

We can represent stabilizers in this format:

# Relationship with stabilizer codes

(Source: Gottesman Thm. 4.4)

More generally, given a linear code  $C$  and parity check matrix  $H$ , define stabilizer code with binary symplectic representation,

This stabilizer code has the same basis codewords as  $C$  (plus linear combinations thereof), and corrects the same set of errors.

# CSS codes

(Source: Gottesman Def. 5.1)

A stabilizer code is a CSS code if there exists a choice of stabilizer generators with binary symplectic representation of the form

Example: choose both  $A$  and  $B$  to be  $H$  of  $[7, 4, 3]$  Hamming code:

# The Steane code

Stabilizer generators:

What are the logical operations? What are the code parameters?



# Relationship with stabilizer codes

(Source: Gottesman Thm. 5.1)

Let  $C_1$  be an  $[n, k_1, d_1]$  classical linear code with parity check matrix  $H_1$ , and  $C_2$  be an  $[n, k_2, d_2]$  classical linear code with parity check matrix  $H_2$ , with  $C_1^\perp \subseteq C_2$ .

The stabilizer code (CSS) code with stabilizer generators

is an  $[[n, k, d]]$  quantum code with  $k = k_1 + k_2 - n$  and  $d \geq \min(d_1, d_2)$ .

# Next time

Next class:

- Some bounds and existence criteria for QECCs
- The perfect code ( $[[5, 1, 3]]$ )

Action items:

- ➊ Prepare for quiz 2
- ➋ Watch PrairieLearn for project information

Recommended reading:

- From this class: Gottesman Ch. 3-4; N&C 10.5
- For next class: Gottesman Ch. 3-4; N&C 10.1-10.5